Algorithms Design and Analysis - HW III

Ronen Finkelstein

Winter 2025-2026

Due to: 18/12/2025

Please submit the assignment in pairs, and make sure to include both of your names and ids on the PDF. 5 bonus points are given for a typed out solution, and my respect to whoever writes in LATEX. Make sure to prove each claim you use, unless it was seen in class. You may contact me for every question you have about the H.W.

Problem 1:

In class, we discussed how to find the diameter of a tree using dynamic programming. In this exercise, we will explore an alternative algorithm that solves the problem using Breadth-First Search (BFS).
Describe an algorithm to find the diameter of a tree using BFS.
Hint: prove that given any vertex w in the tree, at least one of u or v (the endpoints of the diameter) will be the vertex with the maximum distance from w.

Problem 2:

1. For the algorithm BFS shown in class, answer $True/False$ for the following questions, and justify your answers. In every question, $T_{BFS}(G, v)$ denotes the running time of BFS on a graph G starting from a node $v \in V(G)$. Assume $|V(G)| = n$.
   (a) For every graph G and for every $v \in V(G)$, $T_{BFS}(G, v) = O(n^2)$.
   (b) For every $n \geq 2$, there exists a graph G over n vertices, and a vertex v such that $T_{BFS}(G, v) = \Theta(n \log_2 n)$.

2. Recall the running time of BFS on a connected graph G is $\Theta(|V| + |E|)$, provided the graph is given in an adjacency list.
   (a) Now the graph is still connected, but is represented as an adjacency matrix. Give a tight bound for the running time of BFS.
   (b) Give an example for a connected graph G and a vertex v such that

the running time of BFS is asymptotically different for the two ways to represent the graph.
(c) Give an example for a connected graph G and a vertex v such that the running time of BFS is asymptotically equal for the two ways to represent the graph.

3. Answer the following questions and prove your answer.
(a) Give an example for a connected graph over n vertices, such that the order of scanning the graph via BFS is always different than the order of scanning via DFS.
(b) Give an example for a connected graph over n vertices, such that the order of scanning the graph via BFS is always the same as the order of scanning via DFS.

Problem 3:

Let $G = (V, E)$ be a connected (undirected) simple graph. Fix a vertex $s \in V$. Suppose we run Breadth-First Search (BFS) and Depth-First Search (DFS) on G starting at node s. Let T_B and T_D denote the resulting BFS tree and DFS tree, respectively. Prove that if $T_B = T_D$, then G must be a tree.
(Hint: What happens if G has a cycle?)

Problem 4:

A directed graph is called **uniquely connected** if for any two vertices u, v:

1. There is at most one simple path from u to v.
2. There is at most one simple path from v to u.

Describe an algorithm that checks whether a given directed graph is uniquely connected.

Problem 5:

Let $G = (V, E)$ be a directed, unweighted graph and let $s \in V$ be a source vertex.

A vertex $v \in V$ is called **unique-path reachable** from s if there exists *exactly one shortest path* from s to v.
Design an algorithm to find all unique-path reachable vertices from s.

Problem 6:

Let $G = (V, E)$ be a directed graph, where V is the set of vertices and E is the set of directed edges. Given two vertices $u, v \in V$, design an algorithm that detects whether there exists a **non-simple path** from u to v. A **non-simple path** is defined as a path in which at least one vertex is visited more than once (i.e., the path contains repeated vertices or cycles).
Input:

- A directed graph $G = (V, E)$,
- Two vertices u (start vertex) and v (end vertex).

Output: Return **True** if there exists a non-simple path from u to v. Otherwise, return **False**.